



Dynamic priority-based area partitioning, trajectory planning, and task scheduling in computing-while-flying UAV networks

Zijia Zhao^a, Wenhan Zhan^a,^{*} Geyong Min^b, Xu Jiang^a, Liang Zhao^c, Hualong Huang^a

^a School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, 611731, China

^b College of Computer Science, University of Exeter, Exeter, EX4 4QF, UK

^c College of Computer Science, Shenyang Aerospace University, Shenyang, 110136, China

ARTICLE INFO

Keywords:

Mobile edge computing
UAV
Area partitioning
Trajectory planning
Task scheduling
Deep reinforcement learning

ABSTRACT

In Mobile Edge Computing (MEC), Unmanned Aerial Vehicles (UAVs) are emerging as flexible edge servers that address the limitations of traditional static servers. UAV-assisted MEC can deploy edge computing resources to enhance the Quality of Service (QoS) of user devices while dynamically adjusting UAV positions based on real-time demand. Current researches focus on trajectory planning and task scheduling but face challenges like high costs from fixed grid partitioning, load imbalance, and insufficient adaptability in task scheduling. To address these problems, this paper proposes a dynamic priority-based area partitioning, trajectory planning, and task scheduling (MIRATS) scheme in computing-while-flying UAV networks. Specifically, first, we propose an adaptive area partitioning approach that dynamically adjusts UAV numbers and matches them to sub-regions for load balance and lower deployment costs. Second, a task prioritization strategy dynamically adjusts task priorities based on delay constraints and waiting times to ensure a reasonable execution order and position for tasks. Finally, a hierarchical Deep Reinforcement Learning (DRL) framework is designed. The upper DRL optimizes UAV flight paths across sub-regions to reduce flight delay and energy consumption. Meanwhile, the lower DRL dynamically adjusts flight time slots between sub-regions, achieving effective parallel processing between UAV flight and task execution. Extensive experimental results demonstrate that the proposed MIRATS scheme outperforms other schemes by significantly reducing system delay, energy consumption, and deployment costs.

1. Introduction

The rapid development of Internet of Things (IoT) and sixth-generation mobile communications (6G) technologies has given rise to emerging applications such as autonomous driving, immersive media, and virtual reality [1,2]. These applications require computing services with low latency, high density, and strong privacy protection, exceeding the capabilities of current Mobile Users (MUs) [3]. At the same time, the traditional centralized cloud computing model struggles to meet the high-performance requirements of these applications due to network latency and bandwidth limitations [4]. To address this challenge, Mobile Edge Computing (MEC) has emerged as a solution to enhance Quality of Service (QoS) by deploying computing resources closer to users, allowing MUs to offload complex tasks to edge servers [5]. While MEC can enhance service performance, its static deployment of edge servers poses challenges, particularly in scenarios like post-disaster relief or remote areas with inadequate infrastructure [6]. Moreover, even where

deployment is feasible, broader coverage often demands many static servers, which raises costs and limits flexibility in adapting to changing computing needs.

To address the limitations of static edge server deployment, various dynamic deployment schemes have been proposed in recent years. Unlike static methods, dynamic schemes avoid fixed server setups in each region, reducing costs and improving service efficiency by adjusting server locations based on real-time user demands. Among these schemes, UAV-assisted MEC has gained attention for its dynamic deployment capabilities [7]. UAVs are able to provide Line-of-Sight (LoS) communication links, minimize interference from ground obstacles, and reduce communication disruption risks [8]. In addition, UAVs can adjust their positions based on computational loads to improve load balancing and resource utilization. More importantly, their high mobility allows for quick responses to demand changes, enhancing service coverage and lowering deployment costs [9]. Currently, research on UAV-assisted MEC mainly focuses on UAV trajectory optimization

^{*} Corresponding author at: School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, 611731, China.

E-mail addresses: zijiazhao@163.com (Z. Zhao), zhanwenhan@uestc.edu.cn (W. Zhan), g.min@exeter.ac.uk (G. Min), jiangxu617@163.com (X. Jiang), lzhao@sau.edu.cn (L. Zhao), hlhuang_uestc@163.com (H. Huang).

<https://doi.org/10.1016/j.future.2026.108522>

Received 20 August 2025; Received in revised form 5 January 2026; Accepted 10 April 2026

Available online 20 April 2026

0167-739X/© 2026 Published by Elsevier B.V.

and task scheduling to improve computational efficiency and optimize system resource utilization [10].

In UAV-assisted MEC systems, trajectory optimization involves adjusting UAV positions and paths in real-time to improve service coverage and efficiency [11]. Despite various algorithms proposed to optimize UAV trajectories, there are still some challenges. First, existing studies often overlook UAV deployment costs and typically assume a fixed number of UAVs [12]. However, effective trajectory design should dynamically adjust the number of UAVs and allocate them to sub-regions to achieve load balance and minimize costs. In addition, in current UAV-assisted MEC systems, flight and task processing are typically executed sequentially. The UAV must first arrive at a designated hovering position, collect and process tasks while stationary, and only then proceed to the next location [13]. In contrast, if tasks can be processed in parallel during flight, the system delay will be significantly reduced. However, this mode increases the demands on trajectory design, as the UAV must enter each sub-region twice, once for task collection and once for delivering results. The question of how to optimize the flight trajectory and balance the synchronization between task processing and flight still needs to be explored further.

In UAV-assisted MEC systems, the key to task offloading and scheduling lies in efficiently transferring computation tasks from MUs to UAVs and managing them under resource constraints to minimize processing delay and improve responsiveness in dynamic environments [14]. The current task scheduling in UAV-MEC is still facing some challenges. First, the dynamic adjustment of task priority has not been effectively addressed. Existing studies typically assign a fixed priority at the beginning of a task, but in practice, this priority should be adjusted in real time according to the task type, delay constraints, and waiting times [15]. Second, research on task migration is still limited. UAVs can not only directly handle the offloading tasks but also act as relay nodes to pass tasks to ground edge servers when their computation capacity is insufficient [16]. Optimizing relay paths to balance migration costs, network delay, and edge resources remains an urgent challenge.

To address the above challenges, we propose a comprehensive dynamic priority-based area partitioning, trajectory planning, and task scheduling (MIRATS) scheme. The core objective of the scheme is to improve the overall system performance by reducing UAV deployment costs, decreasing task processing delay, and optimizing energy consumption. First, this paper proposes an adaptive area partition method, preventing ineffective flights in low-load regions and resource strain in high-load regions. On this basis, based on the Hungarian algorithm [17], a dynamic UAV matching mechanism is designed to dynamically adjust the number of UAVs and control the deployment costs. Second, a dynamic priority evaluation scheme based on task type, delay constraints, and waiting time is introduced to optimize the task execution order in real time. Finally, hierarchical Deep Reinforcement Learning (DRL) is adopted [18], combined with the innovative strategy of “execution while flying”, to ensure that the UAV efficiently handles tasks during the flight process and realizes the simultaneous optimization of task execution and trajectory planning. The contributions of this paper are as follows.

- An adaptive area partitioning and UAV matching scheme is proposed for load balancing. It dynamically adjusts divisions based on task density and adaptively determines UAV numbers to match sub-regions, reducing deployment costs and improving flight efficiency.
- To ensure the real-time and flexibility of task scheduling, we propose a dynamic task priority evaluation scheme. The scheme takes into account task types, delay constraints, and waiting times to dynamically adjust the priority of tasks, thereby optimizing the execution order and execution position of tasks.

- A joint optimization scheme for UAV trajectory and task scheduling based on a hierarchical DRL framework is proposed. The upper DRL layer is responsible for planning the flight path of the UAVs. The lower DRL layer dynamically adjusts the flight slot length of UAVs between sub-regions to adapt to the “computing-while-flying” mode, and at the same time optimizes the task scheduling strategy in real time to ensure the efficient execution of tasks during the flight process.

The rest of this paper is structured as follows. Section 2 describes the related work. In Section 3, the system model and problem formulation are introduced for clarity. Section 4 outlines the algorithm design and the proposed solutions. Sections 5 and 6 provide the experimental analysis and draw conclusions, respectively.

2. Related work

In this section, the current research on area partitioning and matching in UAV-assisted MEC systems is first introduced. Then, the relevant studies on task prioritization in task scheduling are reviewed. Subsequently, we summarize the recent advances in joint trajectory planning and task offloading. Finally, the shortcomings in the existing studies are systematically sorted out and these studies are synthesized and summarized in Table 1.

2.1. Area partitioning and multi-UAV matching

In trajectory planning, area partition and matching of multiple UAVs are key issues. Depending on the size and complexity of the UAV deployment area, trajectory planning and task execution efficiency can be optimized through reasonable area partition and UAV matching strategies. Akshya et al. [19] proposed a method to divide the area near the obstacles into equal-area rectangles and optimized the path coverage based on this to enhance the efficiency of the task execution. Gui et al. [20] proposed a dynamic center-of-mass area division method to improve the efficiency of task execution through the UAV position to optimize the detection process and ensured that UAVs with fewer tasks are subsequently given more task areas. Chen et al. [21] designed a recursive selection and rate replacement algorithm to optimize the connection between the device and the server through channel quality ranking to achieve integrated optimization of task offloading and movement of UAVs. Han et al. [22] used a particle swarm optimization algorithm to optimize the deployment of UAVs, modeled the user association problem as an optimal transmission problem, and optimized the path coverage based on this.

2.2. Priority computation in task offloading

In recent years, a large number of explorations on priority computation have emerged in task offloading and scheduling. Sharif et al. [23] proposed an adaptive prioritization resource allocation mechanism that flexibly adjusts the resource allocation strategy according to the time constraints of the application. By prioritizing urgent tasks, it improves the overall performance of the system in time-sensitive scenarios. Li et al. [24] used the PPO algorithm to dynamically generate task offloading decisions and priority allocation policies. With this algorithm, the system can adjust the offloading order and priority according to the real-time changes of tasks. Hao et al. [25] investigated a system gain model based on task latency and energy consumption, which classifies tasks into high and low priority. It ensures that critical tasks are completed within strict time constraints, and prevents low-priority tasks from being unexecuted for a long period of time due to resource scarcity. Zhou et al. [26] proposed a system optimization scheme based on task priority by analyzing the computation utility of UAVs in MEC networks, which enhances the efficiency of hotspot selection and task offloading for UAVs.

Table 1
Comparison of solutions for related work.

Ref.	Algorithm	Priority	Considered factors					Optimization objective
			Partition	UAV cost	Collaboration	Trajectory	Scheduling	
[19]	FA/PSO	NA	✓			✓		Find optimal trajectory path.
[20]	NBV	NA	✓		✓			Improve partition performance.
[21]	DDPG	NA	✓			✓	✓	Maximize the system QoE.
[22]	PSO	NA	✓	✓				Minimize the average delay.
[23]	A-PBRA	Static					✓	Maximize resources utilization.
[24]	MEPPO	Dynamic					✓	Minimize delay and energy.
[25]	CLT	Static			✓	✓	✓	Maximize average system gain.
[26]	Q-learning	Static				✓	✓	Maximize the priority utility.
[27]	Auction	NA			✓	✓	✓	Maximize system throughput.
[28]	SARSA	NA				✓	✓	Maximize secrecy rate fairness.
[29]	JDPB	NA	✓			✓	✓	Maximize computing capacity.
[30]	MAAC	NA			✓	✓	✓	Minimize the system cost.
Ours	MADDPG	Dynamic	✓	✓	✓	✓	✓	Minimize delay, energy, cost.

2.3. Joint optimization of trajectory and scheduling

There are various solutions that have been proposed for the joint optimization of trajectory design and task offloading in UAV-assisted MEC. Ning et al. [27] proposed an auction algorithm that determines the UAV trajectory by optimizing the average throughput and adjusts the bids by a community delay approximation algorithm to achieve the co-optimization of the trajectory and the task scheduling. Karmakar et al. [28] devised a method that jointly optimize UAV trajectories, transmission power and task scheduling to maximize secrecy rate and fairness. Zhang et al. [29] proposed a multi-UAV MEC system that jointly optimizes the task offloading, resource allocation and trajectory planning problems, with the goal of maximizing the secure computing power of mobile users. They proposed an algorithm based on dynamic planning and bidding to solve the problem. Gao et al. [30] used the MOIPC-MAAC approach to enhance the overall effect of task allocation and trajectory optimization by designing an IPC network that facilitates communication and collaboration among UAVs.

2.4. Summary

An analysis of existing UAV-assisted MEC systems reveals several common limitations across area partitioning, UAV matching, trajectory design, task offloading, and priority computation. First, partitioning often relies on fixed grids without dynamic load balancing, and UAV-region matching usually fixes the number of UAVs while ignoring deployment costs. Moreover, trajectory designs fail to exploit the parallel processing capability of UAVs during flight. Furthermore, task offloading lacks adaptability to dynamic edge resources, and priority computation remains static, overlooking the evolution of task urgency. To address these challenges, this paper proposes a scheme called MIRATS. In contrast to conventional online optimization methods [31,32], which often rely on accurate models or specific problem structures, these approaches often struggle with real UAV swarms. MIRATS addresses this gap by learning robust policies in dynamic environments, effectively addressing high-dimensional and interconnected optimization challenges related to trajectory planning and task scheduling.

3. System model

In this section, we detail the system model for this paper, including the network, communication, and computation models. In addition, we define the optimization objective and provide relevant mathematical descriptions. All the used notations and parameters are summarized in Table 2.

Table 2
Key notations list.

Notation	Description
M_m	A subregion
$M'_{m,n}$	A user in subregion M_m
$T^t_{m,n}$	The computation task of $M'_{m,n}$
U_k	The k th UAV
C_{es}	Deployment costs of G_{es}
C_k	Deployment costs of U_k
$\alpha'_{k,m}$	Association of U_k and M_m
$\mathcal{M}'_k$	The set of subregions served by U_k
L	Number of subregions served by U_k
P^t_m	Coordinates of hovering position H_m
$P^t_{m,n}$	Coordinates of $M'_{m,n}$
P_{es}	Coordinates of ground edge server G_{es}
$L^t_{m,n,k}$	Average path loss of LoS
$R^t_{m,n,k}$	Communication rate between $M'_{m,n}$ and U_k
$T^{trans}_{m,n,k}(t)$	Transmission delay between $M'_{m,n}$ and U_k
$T^{trans}_{U_k,G}(t)$	Transmission delay between G_{es} and U_k
$T^{hov}_k(t)$	Hovering delay of U_k
$T^{exe}_k(t)$	Execution delay of U_k
$T^{fly}_k(t)$	Flight delay of U_k
$E^{exe}_k(t)$	Execution energy consumption of U_k
$E^{fly}_k(t)$	Flight energy consumption of U_k
T^{total}_k	Total delay of U_k
E^{total}_k	Total energy consumption of U_k
$\Omega(t)$	Total cost at time slot t

3.1. Network model

As shown in Fig. 1, this paper considers a multi-UAV-assisted MEC network. In this model, we assume that the finite region M is divided into M non-overlapping subregions at each time slot t , $M = \{M_1, \dots, M_M\}$. Each subregion has N user requests, and the user arrival process obeys a Poisson distribution with parameter γ . The task for each user $M'_{m,n}$ can be denoted as $T^t_{m,n} = \{I^t_{m,n}, O^t_{m,n}, C^t_{m,n}, \tau^t_{m,n}\}$, where $I^t_{m,n}$, $O^t_{m,n}$, $C^t_{m,n}$, and $\tau^t_{m,n}$ respectively denote the amount of input data, the output data, the number of CPUs, and the task deadline.

Given the high deployment cost and limited coverage of ground edge servers, only one server, denoted as G_{es} , is deployed in the entire region. UAVs are utilized to provide MEC services, not only by directly handling computation tasks but also by serving as relay nodes to forward tasks to the ground edge server. Assuming K UAVs are deployed at time slot t , we represent them as $K = \{U_1, \dots, U_K\}$. The total deployment costs are expressed as

$$C^t_{total} = C_{es} + \sum_{k=1}^K C_k, \quad (1)$$

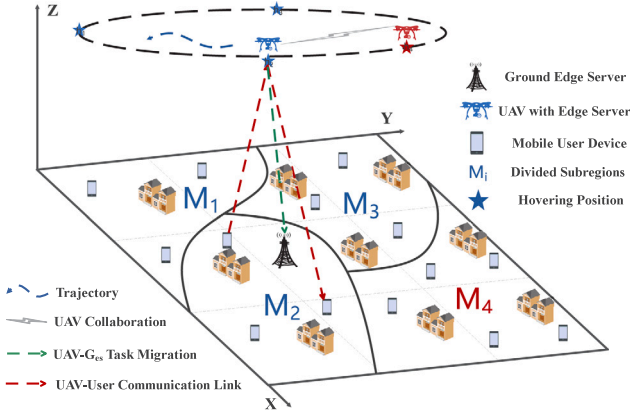


Fig. 1. An illustration of multi-UAV-assisted MEC model.

where C_{es} and C_k denote the deployment costs of the ground edge server and each UAV, respectively. By dynamically adjusting the number of UAVs K , this paper optimizes the deployment costs while satisfying the computation requirements.

In this model, the association between UAVs and subregions at time slot t is defined by a binary variable $\alpha'_{k,m}$. Specifically, $\alpha'_{k,m} = 1$ indicates that UAV U_k is assigned to subregion M_m . Consequently, the set of all subregions served by U_k is denoted as $\mathcal{M}'_k = \{M_1, \dots, M_L\}$, which means that U_k is responsible for trajectory planning and task processing in L subregions. Each M_m has a fixed hovering position H_m for the UAV, which remains at a constant height when hovering. From this position, the UAV can cover the entire M_m and receive task requests from users in the region. Instead of processing all tasks at H_m after receiving them from M_m , U_k can process tasks in parallel while moving to the next subregion. This “computing-while-flying” model enables the UAV to make two passes in each subregion, one to collect tasks and another to return with results. Let $\phi'_{k,m}$ denote the number of associations between U_k and M_m . Therefore, this model is different from the traditional model of trajectory planning. The scenarios in this paper are modeled as a three-dimensional cartesian coordinate system, where the coordinates of $T'_{m,n}$ are $P'_{m,n} = \{x'_{m,n}, y'_{m,n}, 0\}$, the coordinates of H_m are $P'_m = \{x'_m, y'_m, h\}$, and the coordinates of G_{es} are $P_{es} = \{x_{es}, y_{es}, 0\}$.

3.2. Communication model

Assuming the UAV adopts Orthogonal Frequency Division Multiple Access (OFDMA) technology, all users connect to it through OFDMA [33]. This paper employs a probabilistic path loss model [34], considering a line-of-sight (LoS) channel, with average path loss described as

$$L'_{m,n,k} = p'_{m,n,k} \cdot L'_{m,n,k}^{Los} + (1 - p'_{m,n,k}) \cdot L'_{m,n,k}^{NLos}, \quad (2)$$

where $p'_{m,n,k}$ denotes the probability of LoS channel between $M'_{m,n}$ and U_k , which is given by

$$p'_{m,n,k} = \frac{1}{1 + \alpha \cdot \exp[-\beta(\theta'^t_{m,n,k} - \alpha)]}, \quad (3)$$

where $\theta'^t_{m,n,k}$ denotes the elevation angle of U_k relative to $M'_{m,n}$, and α and β reflect the parameters of environmental impact. $L'_{m,n,k}^{Los}$ and $L'_{m,n,k}^{NLos}$ denote the average path loss of the LoS channel and NLoS channel between $M'_{m,n}$ and U_k , respectively. They can be calculated by

$$L'_{m,n,k}^{Los} = 20 \log_{10} \left(\frac{4\pi f_c}{c} \text{dis}^t_{m,n,k} \right) + \xi_{los}, \quad (4)$$

$$L'_{m,n,k}^{NLos} = 20 \log_{10} \left(\frac{4\pi f_c}{c} \text{dis}^t_{m,n,k} \right) + \xi_{nlos}, \quad (5)$$

where the first half of this equation denotes the Free Space Propagation Loss (FSPL). c represents the speed of light and f_c the carrier frequency. ξ_{los} and ξ_{nlos} denote the average additional loss to FSPL for LoS and NLoS, respectively. Thus, the communication rate between $T'_{m,n}$ and U_k at time t is defined as

$$R'_{m,n,k} = B_{m,n,k} \log_2 \left(1 + \frac{P/L'_{m,n,k}}{\delta^2} \right), \quad (6)$$

where P is the transmission power of $M'_{m,n}$, $B_{m,n,k}$ is the channel bandwidth, and δ^2 is the noise power. The downlink data is generally much smaller than the input data and can be ignored [35]. Finally, the transmission delay between $T'_{m,n}$ and U_k can be given by

$$T'_{m,n,k}^{trans}(t) = \frac{I'_{m,n}}{R'_{m,n,k}}. \quad (7)$$

Similarly, the transmission delay between U_k and the ground edge server is expressed by

$$T'_{U_k,G}^{trans}(t) = \frac{I'_{U_k,G}}{R'_{U_k,G}}. \quad (8)$$

3.3. Computing model

The UAV can perform tasks in-flight and transfer them to a ground server. Total delay comprises hovering time, flight delay, and task execution delay. The total hovering delay of U_k in the subregion for which it is responsible is defined as

$$T_k^{hov}(t) = \sum_{l=1}^L \max_{n \in \{1, \dots, N\}} T'_{l,n,k}^{trans}(t), \quad (9)$$

which denotes the maximum upload time of all tasks in the subregion. Suppose that U_k collects N tasks in each subregion, and Q of them choose to migrate out for execution, and the rest are executed on the UAV. The execution delay of U_k on subregion M_l is defined by

$$T'_{l,k}^{exe}(t) = \sum_{n=1}^{N-Q} T'_{l,n,k}^{exe}(t), \quad (10)$$

where $T'_{l,n,k}^{exe}(t)$ denotes the execution delay of $T'_{m,n}$, which can be calculated by

$$T'_{l,n,k}^{exe}(t) = \frac{C'_{l,n}}{f'_{l,k}}, \quad (11)$$

where $f'_{l,k}$ denote the computing power of UAV. The execution delay of the ground server on subregion M_l is defined as

$$T'_{l,G}^{exe}(t) = \sum_{n=N-Q+1}^N (T'_{l,n,G}^{exe}(t) + T'_{U_k,G}^{trans}(t)), \quad (12)$$

where $T'_{l,n,G}^{exe}(t)$ denotes the execution delay for migrating out, which can be calculated by

$$T'_{l,n,G}^{exe}(t) = \frac{C'_{l,n}}{f_{mig}}, \quad (13)$$

where f_{mig} denotes the computing power of the ground server. $T'_{U_k,G}^{trans}(t)$ denotes the transmission delay when migrating. Therefore, the total execution delay of U_k in all subregions is expressed as

$$T_k^{exe}(t) = \sum_{l=1}^L \max \{ T'_{l,k}^{exe}(t), T'_{l,G}^{exe}(t) \}. \quad (14)$$

The execution energy consumption of U_k is computed as

$$E_k^{exe}(t) = \sum_{l=1}^L P_k^{exe} \cdot T'_{l,k}^{exe}(t), \quad (15)$$

where P_k^{exe} is the execution power. The total flight delay of U_k in all the subregions under its responsibility is expressed as

$$T_k^{fly}(t) = \sum_{l=1}^{2L-1} t_l, \quad (16)$$

which is determined dynamically by the DRL algorithm to accommodate the “computing-while-flying” model. Let t_l denote the flight time from subregion M_l to the next subregion, which is influenced by the flight speed of UAV, $v_k \in (0, v_{\max}]$. The flight energy consumption of U_k is described as

$$E_k^{fly}(t) = P_k^{fly} \cdot T_k^{fly}(t), \quad (17)$$

where P_k^{fly} is the flight power. Since UAV flight and task execution can be performed in parallel, the total delay of U_k can be expressed as

$$T_k^{total}(t) = \max\{T_k^{exe}(t), T_k^{hov}(t) + T_k^{fly}(t)\}. \quad (18)$$

The total energy consumption is defined as

$$E_k^{total}(t) = E_k^{fly}(t) + E_k^{exe}(t). \quad (19)$$

3.4. Problem formulation

Based on the above definition, the total system cost depends on the area partitioning ($\mathcal{P}$) and trajectory planning and task scheduling ($\mathcal{T}$). The cost at time slot t is modeled as the weighted sum of UAV deployment costs, delay, and energy consumption, which can be expressed as

$$\Omega(t) = \eta_C \cdot C^{total}(t) + \eta_T \cdot T^{total}(t) + \eta_E \cdot E^{total}(t), \quad (20)$$

where η_C, η_T, η_E are non-negative weighting factors representing the relative importance of deployment cost, delay, and energy consumption, respectively. $C^{total}(t)$ and $E^{total}(t)$ respectively represent the total cost and total energy consumption of all UAVs, and $T^{total}(t)$ denotes the maximum delay of all UAVs. Our goal is to minimize the average system cost over a total of T time slots, which can be formulated as the following optimization problem:

$$P : \min_{\mathcal{P}, \mathcal{T}} \frac{1}{T} \sum_{t=1}^T \Omega(t) \quad (21)$$

$$s.t. \ M_i \cap M_j = \emptyset, \quad (21a)$$

$$M = \bigcup_{k=1}^K \mathcal{M}_k', \quad (21b)$$

$$\sum_{k=1}^K \alpha_{k,m} = 1, \quad (21c)$$

$$\varphi_{k,m} \geq 2, \quad (21d)$$

$$E_k^{total}(t) \leq E_k^{\max}(t), \quad (21e)$$

$$T_{l,n,k}^{trans}(t) + \sum_{q=1}^Q T_{l,q,k}^{exe}(t) + T_{l,n,k}^{exe}(t) \leq \tau_{l,n}, \quad (21f)$$

$$T_{l,n,k}^{trans}(t) + T_{U_k,G}^{trans}(t) + \sum_{q=1}^Q T_{q,G}^{exe}(t) + T_{l,n,G}^{exe}(t) \leq \tau_{l,n}. \quad (21g)$$

Condition (21a) means that there is no overlap between the two subregions. Condition (21b) indicates the regions for which all UAVs are responsible are combined together as the entire region. Condition (21c) means that one subregion can belong to only one UAV. Condition (21d) denotes the UAV flies to the hovering position in each subregion at least twice. Condition (21e) indicates that the total energy consumption of U_k cannot exceed its energy constraint $E_k^{\max}(t)$. Condition (21f) and (21g) represents that the processing delay of each task cannot exceed its delay constraint, here, Q denotes the number of tasks that have already been scheduled on the same server prior to the current task, which is used to estimate the queuing delay.

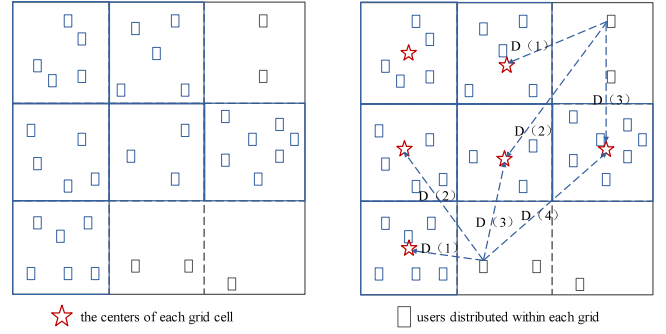


Fig. 2. An example diagram of the adaptive partitioning method.

4. Proposed solution

In this section, as shown in Fig. 3, we present our proposed solution and its each key step in detail. First, we design an adaptive area partitioning and UAV matching scheme. Second, we propose a dynamic task priority computation strategy, which aims to adjust the priority of tasks in real time according to the task type, delay constraints, and waiting time. Finally, we develop a hierarchical DRL framework for the joint optimization of trajectory planning and task scheduling in the “computing-while-flying” mode.

4.1. Adaptive partitioning and matching scheme

In this subsection, we adopt an adaptive area partitioning strategy to balance the load of each subregion, and combine it with an improved Hungarian algorithm to dynamically determine the number of UAVs to achieve efficient matching between UAVs and subregions. This scheme can effectively reduce the deployment costs of the system while ensuring the service quality.

4.1.1. Partitioning strategy based on clustering algorithm

Conventional fixed-grid partitioning often leads to unbalanced loads, causing UAV inefficiencies. These static divisions cannot adapt to task load variations, resulting in some grids being overloaded while others are underutilized. To address this, we propose an adaptive area partitioning strategy based on an improved clustering algorithm to balance loads and optimize UAV trajectories.

As shown in Fig. 2, we assume that the user arrival rate γ in the entire region follows a Poisson distribution, here are the task arrivals in the entire region over T time steps [36]. First, the ground area is partitioned into $I \times J$ rectangular grid cells, denoted as $G(i, j)$, where i and j indicate the row and column indices, respectively. Second, each grid is assigned a task load value $l(i, j)$, defined as the number of tasks within that grid. We introduce a load threshold Z computed as the average task load across all grids, which can be calculated by

$$Z = \frac{\sum_{i=1}^I \sum_{j=1}^J l(i, j)}{I \times J} \cdot Z_{\text{factor}}. \quad (22)$$

Then, the grid is adaptively adjusted according to the load of each grid. Specifically, a grid $G(i, j)$ is treated as an independent service grid if $l(i, j) > Z$. Let H be the total number of such independent grids. On this basis, the centers of these H grids serve as clustering centers. Tasks from sparse grids are reassigned to one of these H centers based on a weighted distance measure. The weighted distance from a task $t_{i,j}^n$ in a sparse grid to the h th clustering center is calculated as

$$D(h) = \phi_1 \cdot d(h, t_{i,j}^n) + \phi_2 \cdot l(h), \quad (23)$$

where $t_{i,j}^n$ denotes a task within grid $G(i, j)$, $d(h, t_{i,j}^n)$ denotes the Euclidean distance from task $t_{i,j}^n$ to the h th clustering center, and $l(h)$

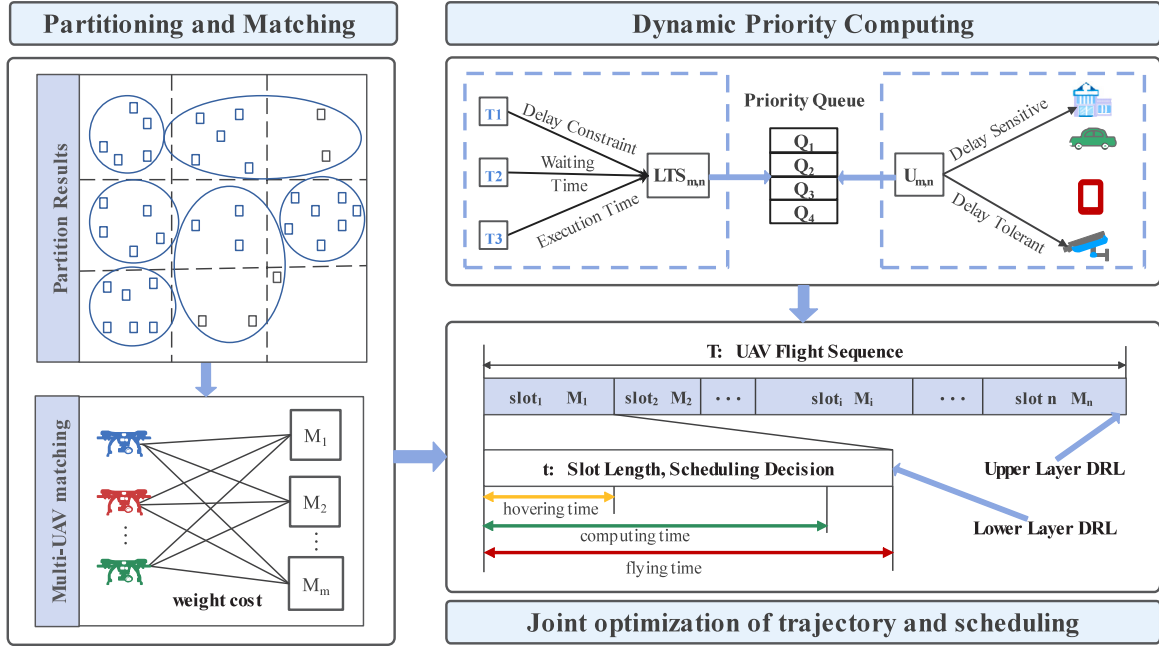


Fig. 3. A comprehensive scheme for MIRATS.

represents the task load of the h th center. This formulation effectively combines proximity-based assignment with load-aware balancing, encouraging tasks to be assigned to nearby and lightly-loaded grids.

4.1.2. Matching strategy based on Hungarian algorithm

After partitioning the regions, the next step is to determine the optimal number of UAVs and match them to subregions. This balance is crucial for minimizing deployment costs and ensuring sufficient task coverage. Matching UAVs to subregions is essentially similar to the NP-hard Set Cover Problem (SCP) [37]. Therefore, we propose a matching strategy based on the Hungarian algorithm to achieve efficient matching and control system deployment costs. The implementation process is detailed below.

First, the number of UAVs is initially set based on the results of the partitioning, that is, each sub-region is set to be equipped with one UAV. This aims to avoid the problem of missing coverage due to insufficient number of UAVs.

Second, the cost matrix C is constructed, and each element of this matrix represents the matching cost between U_k and M_m . The matching cost is determined by the delay required for U_k to compute the tasks in M_m and the distance between U_k and M_m . The formula is defined as

$$C[k][m] = \delta_1 \cdot \sum_{n=1}^N \frac{C_{m,n}}{f_{m,k}} + \delta_2 \cdot d_{m,k}. \quad (24)$$

Then, the Hungarian algorithm is used to match the K UAVs with M subregions. This algorithm is able to find the optimal matching scheme in polynomial time. The goal is to minimize the total cost of all the matches. The final match result is $UM = \{\{UM_1\}, \{UM_2\}, \dots, \{UM_K\}\}$.

Finally, adjust the number of UAVs K' . Gradually reduce the UAV count and recalculate the matching after each reduction to observe cost changes. Since the number of subregions may not match the number of UAVs, virtual columns with high costs are added to the cost matrix. If after matching, any U_k experiences a significant delay that fails to meet the task requirements, the optimal number of UAVs is set to $K' + 1$, and the matching result corresponds to this number. Algorithm 1 describes the detailed process of area partitioning and matching scheme. Algorithm 1 is performed in the following steps. First, the

region grid density and the set of partitions are initialized. Next, the algorithm performs adaptive area partitioning for the entire region. Finally, by optimizing the number of UAVs, the matching association with the sub-regions is achieved. The time complexity of Algorithm 1 is analyzed as follows. The time complexity of calculating the total density of the region and dividing the grid is $O(I \times J)$, the complexity of the clustering process is $O(M' \times d_{max})$, the complexity of computing the cost matrix is $O(M \times K)$, the complexity of the Hungarian algorithm is $O(K^3)$, and the complexity of dynamically adapting the number of UAVs is $O(K^2)$. Therefore, the total time complexity of the whole algorithm is $O(I \times J) + O(M' \times d_{max}) + O(M \times K) + O(K^3)$.

4.2. Dynamic priority computing scheme

In UAV task execution, they usually receive requests from multiple users with different delay constraints. If a task fails to meet its delay requirement, it is considered a failure. Given the limited computation resources of UAVs, optimizing task execution order is crucial to minimize average delay and ensure more tasks meet their deadlines. Traditional prioritization methods use static strategies, which can lead to low-priority tasks being delayed or even starved. To address this, we propose a dynamic priority scheme based on task types, delay constraints, and waiting times, improving fairness in resource allocation and optimizing system performance in multi-task scenarios.

To be specific, each UAV maintains a priority queue Q_k to manage task scheduling. Upon task arrival, tasks are sorted by priority from highest to lowest. When the UAV reaches a partition, all unexecuted tasks are reevaluated, and the queue is adjusted dynamically. This real-time priority recalculation optimizes scheduling by considering task type and waiting time, ensuring long-term fairness and preventing low-priority tasks from being starved. The formula for task priority is given by

$$Q_{m,n} = \eta_1 \cdot LST_{m,n} + \eta_2 \cdot U_{m,n}, \quad (25)$$

where $LST_{m,n}$ denotes the latest start time, which refers to the fact that if the task has not started executing at this moment, the task will not be able to complete within the specified delay constraints, which will

Algorithm 1: Partitioning and Matching Scheme

Input: The arrival task of T timesteps in the entire region.
Output: Partitioning results, optimal number of UAVs, and matching results.

```

1 Initialize the region into  $I \times J$  grids  $G(i, j)$ , grid density  $d(i, j)$ ,
  grid load threshold  $Z$ , the set of independent grids  $G$ , and the
  set of non-independent grids  $G'$ ;
  Adaptive region partitioning
  for  $i = 1$  to  $I$  do
2    for  $j = 1$  to  $J$  do
3       $D+ = d(i, j)$ 
4    end
5  end
6   $Z = 0.75 \cdot D / (I \times J)$ 
  for  $i = 1$  to  $I$  do
7    for  $j = 1$  to  $J$  do
8      if  $d(i, j) > Z$  then
9        Add  $G(i, j)$  to  $G$ 
10     end
11     else
12       Add  $G(i, j)$  to  $G'$ 
13     end
14   end
15 end
16 for  $m' = 1$  to  $M'$  do
17   for  $n = 1$  to  $d(m')$  do
18     Compute  $D(n, m)$  according to Eq. (23)
19      $G(m) = \text{union}(G(m), M_{m',n})$ , where
20      $D(n, m) = \max(D(n, m))$ 
21   end
22 end
23 UAV and region matching
  for  $m = 1$  to  $M$  do
24   for  $k = 1$  to  $K$  do
25     Compute the cost  $C[k][m]$  according to Eq. (24)
26   end
27 end
28 Matching based on Hungarian algorithm
  while  $K > 0$  do
29    $K = K - 1$ 
30   Copy UAV and expand cost matrix
31   Update  $UM = \{UM_1, UM_2, \dots, UM_K\}$ 
32   for  $k = 1$  to  $K$  do
33      $T_k = \sum_{j=1}^J \frac{C_{k,j}}{f_k}$ 
34     if  $T_k > T_{max}$  then
35        $K_{opt} = K + 1$ 
36       exit
37     end
38   end
39 end

```

result in the failure of the task offloading. The smaller its value, the more urgent the task. Its formula is described as

$$LST_{m,n} = \tau_{m,n} - T_{m,n}^{exe} - T_{m,n}^{wait}, \quad (26)$$

where $T_{m,n}^{wait}$ denotes the waiting time, which is dynamically changing. Its computing formula is defined by

$$T_{m,n}^{wait} = \sum_{l=1}^{l'} T_{k,l}^{hov} + T_{k,l}^{fly}. \quad (27)$$

Since the priority of a task is dynamically computed as the UAV arrives at each subregion, the waiting time of the task is closely related to the

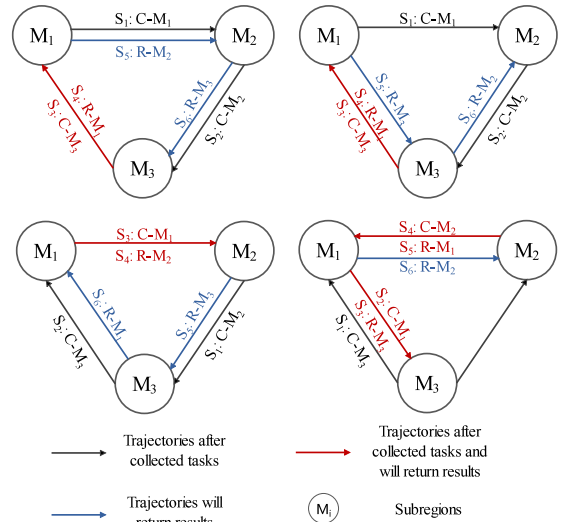


Fig. 4. Example diagrams of different flight trajectories of the UAV.

hovering time on each previous subregion and the flight time between subregions. Specifically, if a task is not processed during UAV flight, its waiting time will increase with the flight slot. This means that the latest start time of a task will be shortened by the increase in the waiting time, thus enhancing the urgency of the task. This mechanism ensures that the priority of a task can reflect the actual urgency of the task in real time.

In task prioritization evaluation, the value of parameter $U_{m,n}$ is closely related to the type of task. Specifically, the value of $U_{m,n}$ is set lower for high-priority tasks and higher for low-priority tasks. Thus, a smaller $Q_{m,n}$ value indicates that the task has a higher priority. This setting ensures that the system is able to prioritize high-priority tasks, thereby improving overall task processing efficiency.

4.3. Joint optimization scheme based on hierarchical DRL

To improve efficiency, this paper introduces a computing-while-flying model for UAVs. Instead of finishing all tasks in one sub-region before moving on, UAVs will fly to the next sub-region after gathering tasks, processing them in parallel during flight intervals. Tasks completed during flight intervals are directly transmitted back to the user, while unfinished tasks are reordered for future scheduling. When a UAV arrives in a region where its tasks remain unfinished, it does not pause. Instead, the completed results are transmitted to the ground edge server for final delivery as soon as they are ready. The transmission delay is negligible since the data volume for computational output is much smaller than for task input [35]. To further optimize this workflow, we propose a hierarchical DRL framework. The upper DRL manages the global flight sequence, while the lower DRL dynamically allocates flight times and schedules tasks in sub-regions. The lower layer guides the agent to learn matching strategies between task cycles and flight times through a reward mechanism. When the UAV returns with all tasks completed, it receives a high reward; otherwise, it receives a low reward. This continuous learning approach enhances flight time allocation and overall task execution efficiency.

4.3.1. Design of the upper SAR

The main goal of the upper layer DRL is to optimize the collection and return sequence of tasks through intelligent flight sequence decision making. Its state space, action space, and reward function are designed as follows.

(1) State space. The state space of the upper DRL can be expressed as

$$S^{upper} = \{\mathcal{M}_k, D_k, N_k\}, \quad (28)$$

where $\mathcal{M}_k = \{M_1, \dots, M_L\}$ denotes the matching results of U_k , $D_k = [d_{l,l'}]_{L \times L}$ denotes the distance matrix between partitions on each matching set, and $N_k = \{N_1, N_2, \dots, N_L\}$ denotes the number of tasks in each region, and $N_l = \{T_1, T_2, \dots, T_T\}$ represents the number of tasks in this region at different time steps.

(2) Action space. The action space of the upper DRL can be expressed as

$$A^{upper} = \{P_k^{collect}, P_k^{return}\}, \quad (29)$$

where $P_k^{collect} = \{M_l^1, M_l^2, \dots, M_l^L\}$ represents the task collection sequence, and $P_k^{return} = \{M_{l'}^1, M_{l'}^2, \dots, M_{l'}^L\}$ represents the task return sequence.

(3) Reward function. The reward function of the upper DRL is related to task accumulation and flight energy consumption. The goal is to minimize the task accumulation and flight energy consumption. Thus, it can be expressed as

$$R^{upper} = -(\gamma_1 \cdot t_k^{acc} + \gamma_2 \cdot E_k^{fly}). \quad (30)$$

As shown in Fig. 4, different path choices result in varying distances, which affect both flight energy consumption and task execution progress in the “computing-while-flying” mode. In our design, energy consumption in the upper DRL is tied to flight distance, while task accumulation depends on task arrival rate and total flight distance. To minimize backlog, the UAV should prioritize collecting tasks that arrive first. Optimizing path selection can reduce both energy consumption and task accumulation, improving overall system efficiency.

4.3.2. Design of the lower SAR

The lower DRL is responsible for dynamically optimizing the flight time and task scheduling of UAVs between regions, ensuring that UAVs can “computing while flying”, minimizing delay and energy consumption. The state space, action space, and reward function of the lower DRL are defined as follows.

(1) State space. The state space of the lower DRL can be expressed as

$$S^{lower} = \{P_k^{collect}, P_k^{return}, Q_k, T_m, R_k, R_{bs}\}, \quad (31)$$

where $P_k^{collect}$ and P_k^{return} represents the flight order of U_k , Q_k represents the dynamic priority queue of U_k , T_m represents the task situation in each partition, R_k represents the remaining computation resource of each UAV, and R_{es} represents the remaining computation resource of the ground edge server.

(2) Action space. The action space of the lower DRL can be expressed as

$$A^{lower} = \{t_l', \beta_{m,n}\}, \quad (32)$$

where t_l' represents the flight slots from partition M_l to $M_{l'}$, $\beta_{m,n}$ represents the scheduling decision for each task.

(3) Reward function. The reward function of the lower DRL is related to the delay and energy consumption of all tasks, which can be expressed as

$$R^{lower} = -(\eta_k^T \cdot T_k^{total} + \eta_k^E \cdot E_k^{total}). \quad (33)$$

In our design, the reward of the lower DRL is inversely proportional to the delay and energy consumption. The smaller the delay and energy consumption, the larger the reward. Moreover, according to (21), it can be observed that the smaller the delay and energy consumption of a single UAV, the overall delay and energy consumption of the system will be reduced accordingly.

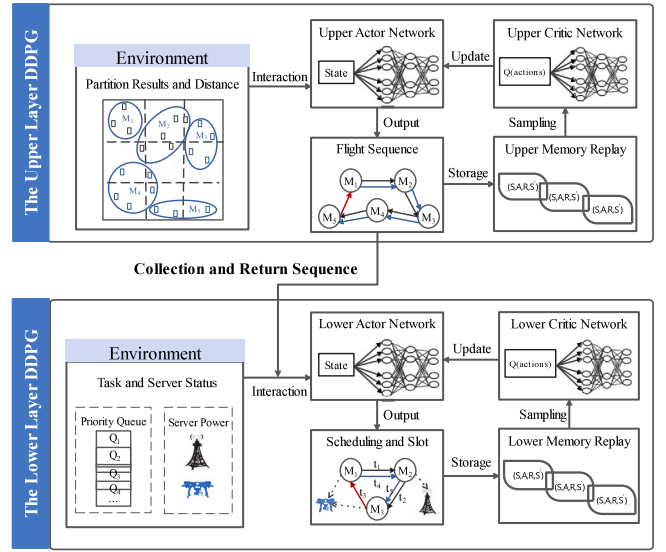


Fig. 5. The hierarchical DRL framework based on DDPG.

4.3.3. The AC framework for the DDPG algorithm

This subsection provides an in-depth training process analysis of the AC framework within the DDPG algorithm. Fig. 5 shows the training process of the DDPG-based hierarchical framework for trajectory planning and task scheduling.

First, the current actor network interacts with the environment. The actor network in the upper layer generates action a_t^{upper} based on the current state s_t^{upper} . After executing the action, the system interacts with the environment to obtain a new state s_{t+1}^{upper} and reward r_t^{upper} . The corresponding experience quaternion $(s_t^{upper}, a_t^{upper}, s_{t+1}^{upper}, r_t^{upper})$ is deposited into the upper layer experience replay pool. Similarly, the lower Actor network interacts with the environment and the generated experience quaternion $(s_t^{lower}, a_t^{lower}, s_{t+1}^{lower}, r_t^{lower})$ is deposited into the lower experience replay pool.

Second, the current critic network is updated. For both upper and lower critic networks, the update process is consistent. We randomly sample experiences from the pool, using the critic network to generate the current Q-value and the target critic network for the target Q-value, calculated as

$$Q_t^{target} = r_t + \gamma Q'(s_{t+1}, \mu'(s_{t+1} | \theta^{\mu'}) | \theta^{Q'}). \quad (34)$$

The parameter of the current critic network is updated by minimizing the error between the current Q-value and the target Q-value, it is updated by

$$Loss = \frac{1}{T} \sum_{i=1}^T (Q_i^{target} - Q(s_i, a_i | \theta^Q))^2. \quad (35)$$

Third, the current actor network is updated. The actor network performs parameter optimization by maximizing the Q-value fed back by the critic network, and its parameter is updated by

$$\nabla_{\theta^{\mu}} J \approx \frac{1}{T} \sum_i \nabla_a Q(s_i, a_i | \theta^Q) \nabla_{\theta^{\mu}} \mu(s_i | \theta^{\mu}). \quad (36)$$

Finally, the target network is softly updated to maintain training stability, using a soft update mechanism for its parameters, as shown in

$$\theta^{\mu'} \leftarrow \tau \theta^{\mu} + (1 - \tau) \theta^{\mu'}, \quad (37)$$

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}. \quad (38)$$

With this soft update approach, the parameters of the target network slowly follow the current network update, which helps to keep the training process steady.

Algorithm 2 outlines the execution process. It begins with parameter initialization, followed by invoking functions for experience replay and parameter updates. The upper DRL component then handles trajectory planning, while the lower DRL component manages dynamic time slots and task scheduling. In the DDPG algorithm, the computational complexity of a single iteration is $O\left(\sum_{x=1}^X n_x n_{x+1} + \sum_{y=1}^Y n_y n_{y+1}\right)$, representing the total connections between neurons in the actor and critic networks.

Algorithm 2: MIRATS Trajectory and Scheduling Scheme

Input: Matching results between UAVs and subregions.
Output: UAV trajectory, flight slots, scheduling decisions.

- 1 Initialize actor network, critic network, target actor network, and target critic network with parameters $\theta^\mu, \theta^Q, \theta^{\mu'}, \theta^{Q'}$; Initialize replay buffer $B = 10000$, episode length $T = 1000$, discount factor γ , soft update rate τ , and batch size d ;
- Generic experience replay and update**
 if B is full **then**
 - 2 Sample mini-batch from B
 - Compute target Q value using Eq. (34)
 - Update critic network parameters using Eq. (35)
 - Update actor network parameters using Eq. (36)
 - if** $t \bmod d = 0$ **then**
 - 3 Perform soft update on target networks using Eq. (36) and Eq. (37)
 - 4 **end**
- 5 **end**
- 6 **Upper Layer: UAV trajectory planning**
 for episode = 1 **to** 1000 **do**
 - 7 Initialize initial state s_0^{upper}
 - for** $t = 1$ **to** T **do**
 - 8 Select action a_t^{upper} from actor network
 - Execute a_t^{upper} and receive reward r_t^{upper}
 - Acquire the next state s_{t+1}^{upper}
 - Store $(s_t^{upper}, a_t^{upper}, r_t^{upper}, s_{t+1}^{upper})$ into B
 - Call the generic update function
 - 9 **end**
- 10 **end**
- 11 **Lower Layer: task scheduling and dynamic slot**
 for episode = 1 **to** 1000 **do**
 - 12 Initialize initial state s_0^{lower}
 - for** $t = 1$ **to** T **do**
 - 13 Select action a_t^{lower} from actor network
 - Execute a_t^{lower} and receive reward r_t^{lower}
 - Acquire the next state s_{t+1}^{lower}
 - Store $(s_t^{lower}, a_t^{lower}, r_t^{lower}, s_{t+1}^{lower})$ into B
 - Call the generic update function
 - 14 **end**
- 15 **end**

5. Simulation and discussion

In this section, we systematically evaluate the performance of the MIRATS scheme through a large number of simulation experiments. First, the parameter settings of the experiments and their rationality are described in detail. Next, the selected comparison schemes and metrics are described. Finally, the performance of our scheme is compared and analyzed with other schemes in order to comprehensively verify the effectiveness of the MIRATS scheme.

Table 3

Main parameters setting

Parameter	Notation	Value
Size of target area	M	200 m $\times$ 200 m
UAV flight speeds	V_{UAV}	(20, 50) m/s
UAV flight heights	h	100 m
UAV battery capacity	E_{UAV}	500 kJ
Task arrival rate	γ	(0.8, 1.6)
The size of task input data	$I_{m,n}$	(0.8, 1.2) MB
CPU required for the task	$C_{m,n}$	(0.5, 1.0) G
Edge server computing power	f_{ges}	(2.0, 2.8) GHz
UAV computing power	f_{UAV}	(0.8, 1.5) GHz
Delay constraints	$\tau_{m,n}$	(0.8, 1.5) s
MU upload power	P	0.1 W
Channel bandwidth	B	10 MHz
Noise power	δ	-174 dBm/Hz
Discount factor	λ	0.99
Experience pool size	e	6400
Batch size	b	64
Learning rate	α, β	0.0001, 0.001

5.1. Environment setup

Our simulation scenario is designed based on existing studies and the simulation platform uses Python 3.7. First, the target area is set as 200 m $\times$ 200 m and divided into a 4 $\times$ 4 fixed grid, based on which adaptive area partitioning is performed. A ground edge server is deployed in the whole area with computation power (2, 2.8) GHz. Second, the number of UAVs is dynamically varied with the flight speed of (20, 50) m/s, the flight altitude of 100 m, the computation power of UAV is (0.8, 1.5) GHz, and the battery capacity of 500 kJ. Then, the tasks are generated by the MUs, the task arrival rate conforms to the Poisson distribution, the data size of each task is (0.8, 1.2) MB, the computation resource requirement is (0.5, 1) G, the delay constraint is (0.8, 1.5) s, the transmission power of the MU is 0.1 W, and the channel bandwidth is 10 MHz. The weighting factors for the delay, cost, and energy objectives in the objective function are set to $\eta_T = 0.5$, $\eta_C = 0.3$, and $\eta_E = 0.2$. We set $Z_{factor} = 75\%$ through a series of parameter studies, testing from 50% to 90%. Finally, in the DDPG algorithm, a four-layer neural network architecture is constructed, and tanh is chosen as the activation function. In order to further optimize the system performance and accelerate the convergence of the algorithm, the experience playback pool is set to 6400, and the batch size is 64. Additionally, the learning rates of the actor network and the critic network are respectively set to 0.0001 and 0.001. The specific parameter settings can be referred to Table 3.

5.2. Comparison schemes and performance metrics

To validate the effectiveness of MIRATS, we compare it with five schemes: ST-RATS, MIRT, MIRA-SP, MIRA-DQN [38], and MIRA-GA [16]. These schemes address all key aspects of our study for a comprehensive evaluation. Consistent parameters ensure fairness.

(1) ST-RATS. This scheme does not consider dynamic priority adjustment, i.e., it does not dynamically adjust the priority according to the task execution process and waiting time, but only initially sets the priority based on the task type. In contrast, MIRATS adapts to dynamic environments through dynamic priority computation, effectively alleviating task starvation and optimizing processing order in congested scenarios, thereby reducing overall latency.

(2) MIRT. This scheme uses traditional fixed grid division and a defined number of UAVs, ignoring the adaptive partitioning mechanism based on load balancing. In contrast, MIRATS optimizes UAV deployment through adaptive area partitioning, significantly reduces flight energy consumption, and lowers deployment costs.

(3) MIRA-SP. The scheme does not support the computing-while-flying computation mode, but uses a traditional method where the UAV

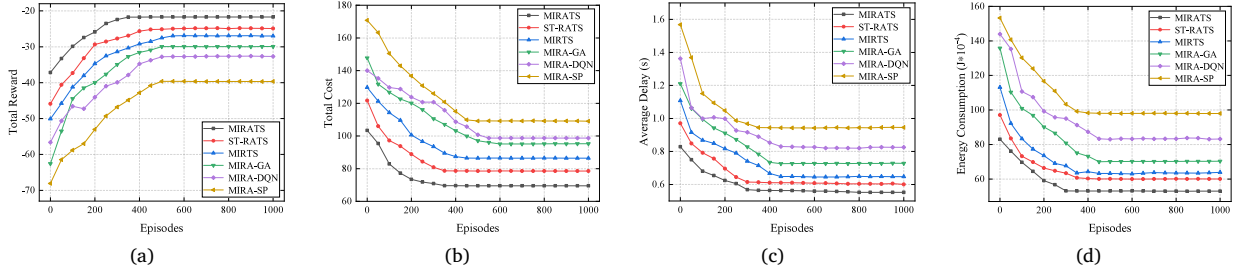


Fig. 6. Comparison of different schemes in terms of (a) TR and (b) TC (c) AD and (d) EC across varying episodes.

collects tasks first and then flies to the next area after completing them. By comparison, the parallel mode of computing-while-flying in MIRATS significantly reduces the task delay and system energy consumption, proving its higher computation efficiency.

(4) MIRA-DQN. This scheme uses a DQN-based task scheduling algorithm. Experiments show that DDPG is more suitable for task scheduling and trajectory planning in continuous action space, which can more reasonably optimize the execution path of tasks and reduce the computation delay and energy consumption of the system.

(5) MIRA-GA. This scheme employs genetic algorithm for task scheduling, based on an iterative mechanism of evolutionary optimization. Although the genetic algorithm performs well in global search capability, the DRL approach can better cope with complex dynamic environments and show greater advantages when facing the problem of trajectory planning and task scheduling for multiple UAVs.

In order to comprehensively evaluate the performance of different schemes, we select the following core performance metrics for analysis.

(1) Average Delay (AD) measures the efficiency of the MEC system by indicating the average time from task submission to completion. Lower latency signifies faster execution and improved responsiveness, showcasing the effectiveness of the algorithm in optimizing resources and scheduling tasks.

(2) Energy Consumption (EC) refers to the total energy consumption of all UAVs during the execution of tasks and flight. As a critical factor in MEC systems, it directly impacts UAV endurance and task continuity. Therefore, optimizing energy consumption is essential for enhancing system stability and resource efficiency.

(3) Total Cost (TC) represents the weighted sum of the deployment costs, energy consumption, and maximum delay of all UAVs in the system. TCC is the core objective of system optimization. A lower TCC implies more effective resource allocation and cost control, reflecting the superiority of the scheme in overall performance.

(4) Success Rate (SR) is used to assess the reliability of the system and is defined as the ratio of the number of successfully completed tasks to the total number of tasks under the delay constraint. A higher SR indicates stronger robustness and stable performance under varying conditions.

(5) Total Reward (TR) denotes the cumulative reward value earned by each UAV during training, reflecting the learning efficiency and adaptability of the algorithms. Higher TR values and faster convergence reflect better environmental adaptability and performance optimization.

5.3. Performance evaluation

This subsection analyzes the proposed MIRATS scheme through comprehensive comparisons. Fig. 6 presents performance comparisons across metrics, Fig. 7 examines UAV computation power and delay constraints, and Fig. 8 shows delay analysis under varying parameters. For reliability, each experiment was independently repeated 10 times under identical conditions, with results averaged every 50 runs to enhance data accuracy.

Fig. 6(a) illustrates the TR performance of six schemes over 1000 episodes. Initially, all schemes show gradual improvement due to iterative optimization. Among them, MIRATS exhibits a notable advantage, converging around the 400th episode, whereas other schemes take longer. This superiority is attributed to its dynamic priority strategy and region partitioning mechanism, which enhance UAV deployment efficiency and state space processing. Furthermore, MIRATS leverages the DDPG algorithm's deterministic policy and AC architecture, enabling superior handling of continuous actions and significantly outperforming MIRA-DQN. In contrast, MIRA-SP consistently achieves the lowest total reward due to its inability to manage flight and mission execution simultaneously, leading to higher latency and energy consumption. Overall, MIRATS demonstrates the best performance in terms of both total reward and convergence speed.

Fig. 6(b) presents the TC trend of the six schemes during the training process of 1000 episodes. It can be seen from the figure that the total cost of each scheme gradually decreases as the number of episodes increases and eventually stabilizes. This decreasing trend is attributed to the fact that the DRL algorithm gradually optimizes the decision-making strategy in the continuous interaction with the environment, which in turn effectively reduces the total cost. MIRATS consistently outperforms all compared schemes due to several key advantages. First, its dynamic priority policy and adaptive region partitioning method optimize the task scheduling order and UAV deployment cost, thus outperforming ST-RATS and MIRTHS in TC. Second, by combining DQN's Q-value estimation with the Actor-Critic's policy gradient, MIRATS excels at handling continuous action problems, surpassing MIRA-DQN and MIRA-GA. Finally, its computing-while-flying mechanism optimizes task scheduling and flight planning, thus making MIRATS superior to MIRA-SP in terms of overall cost optimization.

Fig. 6(c) compares the AD of different schemes over training. While all schemes show decreasing AD, their performance varies significantly. MIRATS achieves the lowest AD due to its optimized task scheduling, parallel execution, and flexible region division. The ST-RATS scheme has a slightly higher delay than MIRATS due to its suboptimal prioritization strategy. The disadvantage of the MIRTHS scheme is mainly due to its failure to realize area division and adaptive scheduling, which leads to lower efficiency of task execution in complex environments. MIRA-GA and MIRA-DQN have higher delays because their decision-making algorithms are less efficient than DDPG, especially in handling successive actions. MIRA-SP experiences the highest delay, as it lacks parallel processing of flights and tasks, significantly increasing task completion time. Overall, the MIRATS scheme, through its optimized task scheduling mechanism, parallel execution strategy, and flexible region division, outperforming all other schemes.

Fig. 6(d) demonstrates the performance comparison of different schemes in terms of EC. Over 1000 episodes, the energy consumption of each scheme shows a decreasing trend and eventually converges to a certain fixed value. Among them, the MIRATS scheme achieves the lowest EC due to its joint optimization strategy, which refines UAV flight paths, minimizes unnecessary flights, and optimizes task scheduling. In contrast, ST-RATS and MIRTHS fail to effectively optimize

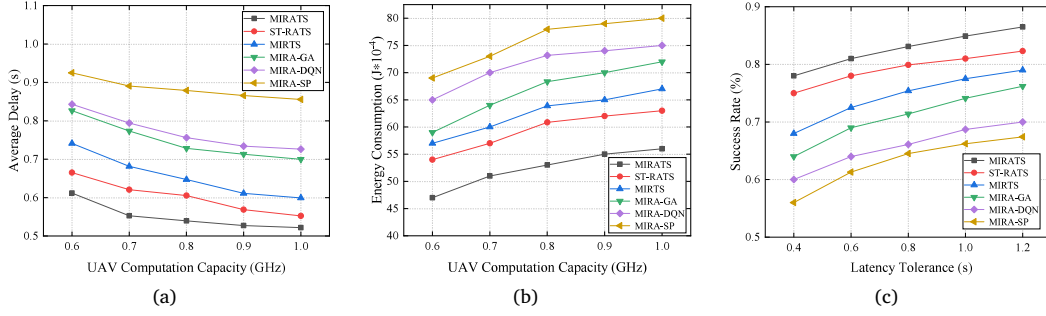


Fig. 7. (a) AD and (b) EC under different UAV computation capacity, (c) SR under different delay constraints.

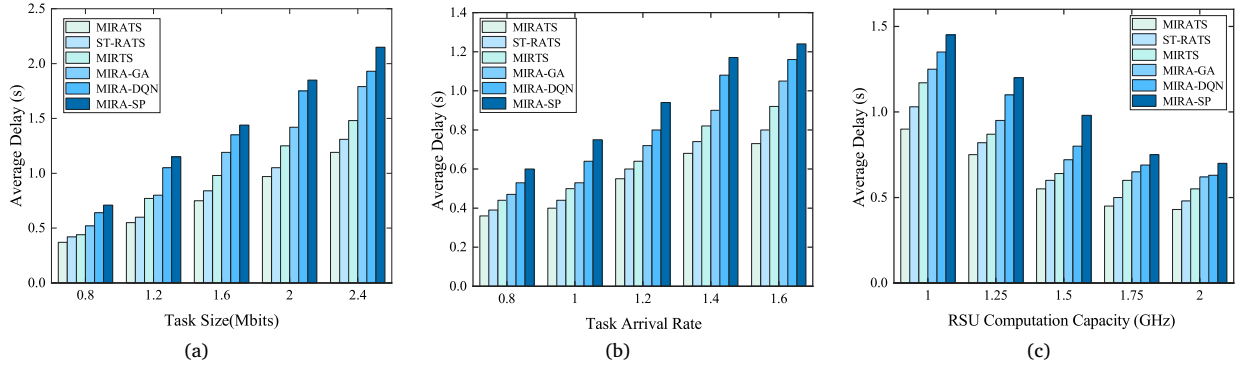


Fig. 8. (a) AD with different task size. (b) AD with different task arrival rate. (c) AD with different UAV computation capacity.

task sequencing and area division, leading to redundant flights and inefficient resource utilization, resulting in significantly higher EC. Additionally, MIRA-GA and MIRA-DQN exhibit higher EC than MIRTH. The poor performance of MIRA-GA is mainly due to the genetic algorithm's slow convergence when facing large search spaces, limiting EC optimization. Meanwhile, MIRA-DQN struggles with EC reduction due to the under-sampling issue in DQN when handling high-dimensional continuous action spaces, affecting UAV path planning and decision-making. These results highlight the advantages of the MIRATS scheme in EC optimization, making it the most efficient among the compared approaches.

The Fig. 7(a) and (b) demonstrate the trends of AD and EC under different UAV computation capability. The results show that the AD tends to decrease and the EC tends to increase with the increase of computation capability, and this trend is more obvious in the early stage and tends to stabilize in the later stage. The specific analysis is as follows. First, regarding the AD, when the computation capability is low, the UAV takes a longer time to complete the task, resulting in a longer task execution time and flight time. As the computation capability increases, the task processing speed is significantly accelerated, which effectively reduces the task execution time and thus reduces the delay. When the computation capability reaches a certain level, the task processing efficiency is close to optimal, and the reduction of delay tends to slow down. Second, regarding the EC, with the increase in computation capability, UAVs are able to handle more computation tasks or more complex tasks, which directly leads to the increase in computation energy consumption. For example, employing high-performance computing units consumes more power to perform tasks. However, with the number of tasks remaining constant, the trend of energy consumption gradually levels off when the computation capacity reaches a high level. In conclusion, the increase in computation capability can effectively optimize delay, but it also brings about an increase in energy consumption, and the delay and energy consumption tend to stabilize with the further increase in computation capability.

Fig. 7(c) illustrates the SR of each scheme under varying delay constraints, including an extended range of 0.4 to 1.2 s to assess performance under stricter real-time requirements. As expected, the SR of all schemes improves with relaxed constraints, as more tasks can be completed within the allotted time. Notably, under highly constrained conditions, such as 0.4 to 0.6 s, MIRATS maintains a significantly higher SR and exhibits the most gradual performance decline. This demonstrates the robustness of its dynamic task scheduling and trajectory optimization mechanisms in preserving task completion reliability even under stringent delay demands, highlighting its suitability for delay-critical applications.

Fig. 8(a) evaluates the AD under different task sizes, with the tested range extended up to 2.4 MB to emulate more data-intensive applications. While all schemes experience increased delay with larger task sizes due to higher computational loads, MIRATS consistently achieves the lowest AD and shows the slowest growth trend, even as task sizes expand beyond 2 MB. This scalability stems from its integrated approach: intelligent region partitioning minimizes redundant UAV movements, dynamic priority allocation ensures timely handling of demanding tasks, and joint trajectory-task scheduling optimizes overall execution efficiency. These attributes collectively enable MIRATS to maintain superior performance under heavier computational workloads, confirming its applicability to real-world data-intensive scenarios.

Fig. 8(b) compares AD under different task arrival rates. It can be seen that the AD of each scheme tends to increase as the task arrival rate increases. This is because, as task arrival rates increase, system load rises, requiring more UAVs and leading to higher latency. It can be seen that the MIRATS scheme consistently outperforms the other schemes, especially at high task arrival rates. This indicates that the MIRATS scheme has excellent scalability and adaptability, and is able to significantly reduce the average delay through efficient task scheduling, trajectory design, and dynamic priority computation when the task volume increases. In contrast, the other schemes fail to effectively cope

with the load variations caused by high task arrival rates, resulting in irrational UAV trajectories and resource scheduling, which increases the system delay.

The Fig. 8(c) shows AD performance under different ground edge server computing capacities. With the improvement of computing power, the AD of each scheme shows a decreasing trend, as task execution and UAV flight time are reduced. Among all the schemes, the MIRATS scheme has the best performance in terms of delay, showing its advantages in resource scheduling and flight path optimization. In contrast, the MIRA-SP scheme has the highest latency because it adopts the strategy of separating the task execution from the UAV flight, resulting in a significant increase in latency. In summary, with the improvement of the computing power of the ground edge server, the efficiency of task execution is significantly improved, and the system latency is reduced as a whole.

6. Conclusion

This paper proposes a scheme called MIRATS, which introduces a novel area partitioning, trajectory planning, and task scheduling strategy for the UAV-assisted MEC system, aiming to optimize the deployment costs, delay, and energy consumption of UAVs. First, we dynamically adjust the area partitioning and the number of UAVs based on the task load to reduce the UAV deployment costs and optimize the UAV flight trajectory. Second, a dynamic priority computation method based on the task execution schedule is proposed to adjust the priority in real time to enhance the scheduling efficiency. Finally, a computation mode of computing-while-flying is proposed, and the trajectory and task execution of UAVs are optimized by the hierarchical DRL algorithm. Simulation results show that MIRATS demonstrates significant advantages in terms of delay, energy consumption, task offloading success rate, and deployment costs, proving its effectiveness. In the future, we will further expand this research and explore more challenging directions, such as UAV collaboration.

CRedit authorship contribution statement

Zijia Zhao: Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Wenhan Zhan:** Writing – review & editing, Supervision, Conceptualization. **Geyong Min:** Visualization, Validation, Resources. **Xu Jiang:** Software, Data curation. **Liang Zhao:** Validation, Supervision. **Hualong Huang:** Methodology, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

References

- [1] X. Chen, M. Li, H. Zhong, X. Chen, Y. Ma, C.-H. Hsu, FUNOff: Offloading applications at function granularity for mobile edge computing, *IEEE Trans. Mob. Comput.* 23 (2) (2023) 1717–1734.
- [2] H. Huang, W. Zhan, G. Min, Z. Duan, K. Peng, Mobility-aware computation offloading with load balancing in smart city networks using MEC federation, *IEEE Trans. Mob. Comput.* (2024).
- [3] G. Sun, Z. Wang, H. Su, H. Yu, B. Lei, M. Guizani, Profit maximization of independent task offloading in mec-enabled 5g internet of vehicles, *IEEE Trans. Intell. Transp. Syst.* (2024).
- [4] L. Zhao, T. Li, E. Zhang, Y. Lin, S. Wan, A. Hawbani, M. Guizani, Adaptive swarm intelligent offloading based on digital twin-assisted prediction in VEC, *IEEE Trans. Mob. Comput.* (2023).
- [5] S. Duan, F. Lyu, H. Wu, W. Chen, H. Lu, Z. Dong, X. Shen, MOTO: Mobility-aware online task offloading with adaptive load balancing in small-cell MEC, *IEEE Trans. Mob. Comput.* 23 (1) (2022) 645–659.
- [6] N. Lin, L. Bai, A. Hawbani, Y. Guan, C. Mao, Z. Liu, L. Zhao, Deep reinforcement learning-based computation offloading for servicing dynamic demand in multi-UAV-assisted IoT network, *IEEE Internet Things J.* (2024).
- [7] L. Zhao, S. Li, Y. Guan, S. Wan, A. Hawbani, Y. Bi, M. Guizani, Adaptive Multi-UAV trajectory planning leveraging digital twin technology for urban IIoT applications, *IEEE Trans. Netw. Sci. Eng.* (2023) 1–16.
- [8] J. Zheng, J. Zhang, B. Ai, UAV communications with WPT-Aided cell-free massive MIMO systems, *IEEE J. Sel. Areas Commun.* 39 (10) (2021) 3114–3128.
- [9] L. Zhao, S. Li, Z. Tan, A. Hawbani, S. Timotheou, K. Yu, A Multi-UAV cooperative task scheduling in dynamic environments: Throughput maximization, *IEEE Trans. Comput.* (ISSN: 1557-9956) (01) (2024) 1–13.
- [10] Y. Gao, X. Yuan, D. Yang, Y. Hu, Y. Cao, A. Schmeink, UAV-assisted MEC system with mobile ground terminals: DRL-Based joint terminal scheduling and UAV 3D trajectory design, *IEEE Trans. Veh. Technol.* (2024).
- [11] M. Zhao, R. Zhang, Z. He, K. Li, Joint optimization of trajectory, offloading, caching, and migration for UAV-assisted MEC, *IEEE Trans. Mob. Comput.* (2024) 1–18.
- [12] Z. Yang, S. Bi, Y.-J.A. Zhang, Online trajectory and resource optimization for stochastic UAV-enabled MEC systems, *IEEE Trans. Wirel. Commun.* 21 (7) (2022) 5629–5643.
- [13] Y. Zeng, S. Chen, Y. Cui, J. Yang, Y. Fu, Joint resource allocation and trajectory optimization in UAV-enabled wirelessly powered MEC for large area, *IEEE Internet Things J.* 10 (17) (2023) 15705–15722.
- [14] L. Zhao, Z. Zhao, A. Hawbani, Z. Liu, Z. Tan, K. Yu, Dynamic caching Dependency-Aware task offloading in mobile edge computing, *IEEE Trans. Comput.* (2025).
- [15] X. Dai, Z. Xiao, H. Jiang, J.C. Lui, UAV-assisted task offloading in vehicular edge computing networks, *IEEE Trans. Mob. Comput.* 23 (4) (2023) 2520–2534.
- [16] J. Tian, D. Wang, H. Zhang, D. Wu, Service satisfaction-oriented task offloading and UAV scheduling in UAV-enabled MEC networks, *IEEE Trans. Wirel. Commun.* 22 (12) (2023) 8949–8964.
- [17] W. Qi, Q. Song, L. Guo, A. Jamalipour, Energy-efficient resource allocation for UAV-assisted vehicular networks with spectrum sharing, *IEEE Trans. Veh. Technol.* 71 (7) (2022) 7691–7702.
- [18] L. Zhao, Z. Zhao, E. Zhang, A. Hawbani, A.Y. Al-Dubai, Z. Tan, A. Hussain, A digital twin-assisted intelligent partial offloading approach for vehicular edge computing, *IEEE J. Sel. Areas Commun.* 41 (11) (2023) 3386–3400.
- [19] P. Priyadarsini, et al., Area partitioning by intelligent UAVs for effective path planning using evolutionary algorithms, 2021, pp. 1–6.
- [20] J. Gui, T. Yu, B. Deng, X. Zhu, W. Yao, Decentralized multi-UAV cooperative exploration using dynamic centroid-based area partition, *Drones* 7 (6) (2023) 337.
- [21] J. Chen, P. Yang, S. Ren, Z. Zhao, X. Cao, D. Wu, Enhancing AIoT device association with task offloading in aerial MEC networks, *IEEE Internet Things J.* (2023).
- [22] Z. Han, T. Zhou, T. Xu, H. Hu, Joint user association and deployment optimization for delay-minimized UAV-aided MEC networks, *IEEE Wirel. Commun. Lett.* (2023).
- [23] Z. Sharif, L.T. Jung, I. Razzak, M. Alazab, Adaptive and priority-based resource allocation for efficient resources utilization in mobile-edge computing, *IEEE Internet Things J.* 10 (4) (2021) 3079–3093.
- [24] P. Li, Z. Xiao, X. Wang, K. Huang, Y. Huang, H. Gao, Eptask: Deep reinforcement learning based energy-efficient and priority-aware task scheduling for dynamic vehicular edge computing, *IEEE Trans. Intell. Veh.* (2023).
- [25] H. Hao, C. Xu, W. Zhang, S. Yang, G.-M. Muntean, Joint task offloading, resource allocation, and trajectory design for multi-uav cooperative edge computing with task priority, *IEEE Trans. Mob. Comput.* (2024).
- [26] W. Zhou, L. Fan, F. Zhou, F. Li, X. Lei, W. Xu, A. Nallanathan, Priority-aware resource scheduling for UAV-mounted mobile edge computing networks, *IEEE Trans. Veh. Technol.* 72 (7) (2023) 9682–9687.
- [27] Z. Ning, P. Dong, M. Wen, X. Wang, L. Guo, R.Y. Kwok, H.V. Poor, 5G-enabled UAV-to-community offloading: Joint trajectory design and task scheduling, *IEEE J. Sel. Areas Commun.* 39 (11) (2021) 3306–3320.
- [28] R. Karmakar, G. Kaddoum, O. Akhrif, A novel federated learning-based smart power and 3D trajectory control for fairness optimization in secure UAV-assisted MEC services, *IEEE Trans. Mob. Comput.* (2023).
- [29] Y. Zhang, Z. Kuang, Y. Feng, F. Hou, Task offloading and trajectory optimization for secure communications in dynamic user Multi-UAV MEC systems, *IEEE Trans. Mob. Comput.* (2024).
- [30] Z. Gao, J. Fu, Z. Jing, Y. Dai, L. Yang, MOIPC-MAAC: Communication-Assisted Multi-Objective MARL for trajectory planning and task offloading in Multi-UAV assisted MEC, *IEEE Internet Things J.* (2024).
- [31] G. Li, J. Cai, An online incentive mechanism for collaborative task offloading in mobile edge computing, *IEEE Trans. Wirel. Commun.* 19 (1) (2019) 624–636.
- [32] Y. Yang, Y. Shi, C. Yi, J. Cai, J. Kang, D. Niyato, X. Shen, Dynamic human digital twin deployment at the edge for task execution: A two-timescale accuracy-aware online optimization, *IEEE Trans. Mob. Comput.* 23 (12) (2024) 12262–12279.

- [33] Z. Hu, F. Zeng, Z. Xiao, B. Fu, H. Jiang, H. Chen, Computation efficiency maximization and QoE-provisioning in UAV-enabled MEC communication systems, *IEEE Trans. Netw. Sci. Eng.* 8 (2) (2021) 1630–1645.
- [34] F. Pervez, A. Sultana, C. Yang, L. Zhao, Energy and latency efficient joint communication and computation optimization in a multi-UAV assisted MEC network, *IEEE Trans. Wirel. Commun.* (2023).
- [35] J. Zhang, X. Xu, S. Han, K. Zhang, P. Zhang, S. Ren, Intelligent Ultra-Reliable and low latency communications: Security and flexibility, *IEEE Trans. Wirel. Commun.* 22 (11) (2023) 8392–8406.
- [36] B. Han, V. Sciancalepore, Y. Xu, D. Feng, H.D. Schotten, Impatient queuing for intelligent task offloading in multiaccess edge computing, *IEEE Trans. Wirel. Commun.* 22 (1) (2022) 59–72.
- [37] R. He, Y. Liu, Y. Xiao, X. Lu, S. Zhang, Deep spatio-temporal 3D densenet with multiscale ConvLSTM-Resnet network for citywide traffic flow forecasting, *Knowl.-Based Syst.* 250 (2022) 109054.
- [38] X. Wei, L. Cai, N. Wei, P. Zou, J. Zhang, S. Subramaniam, Joint UAV trajectory planning, DAG task scheduling, and service function deployment based on DRL in UAV-empowered edge computing, *IEEE Internet Things J.* 10 (14) (2023) 12826–12838.